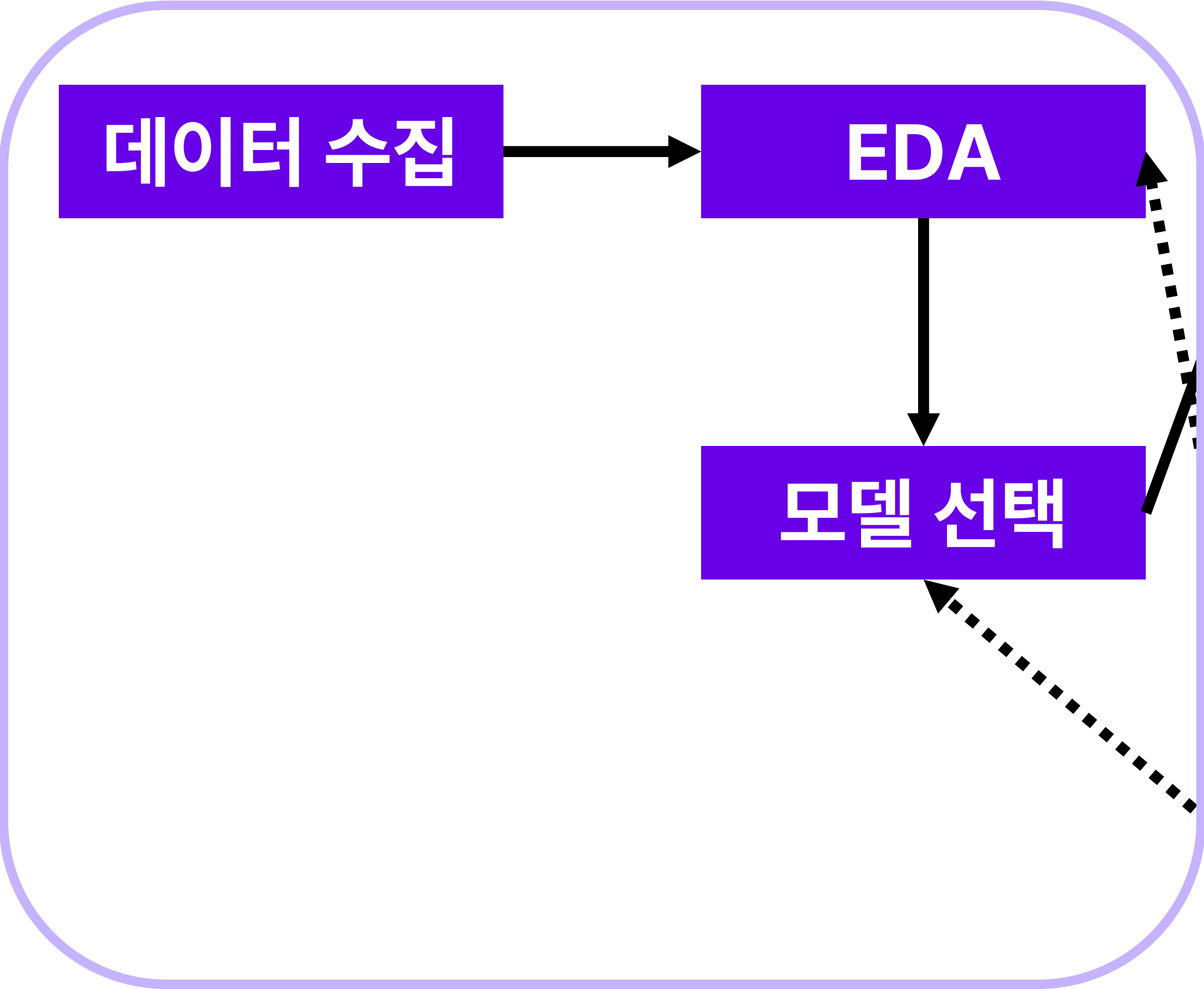


머신러닝 프로젝트 진행 과정

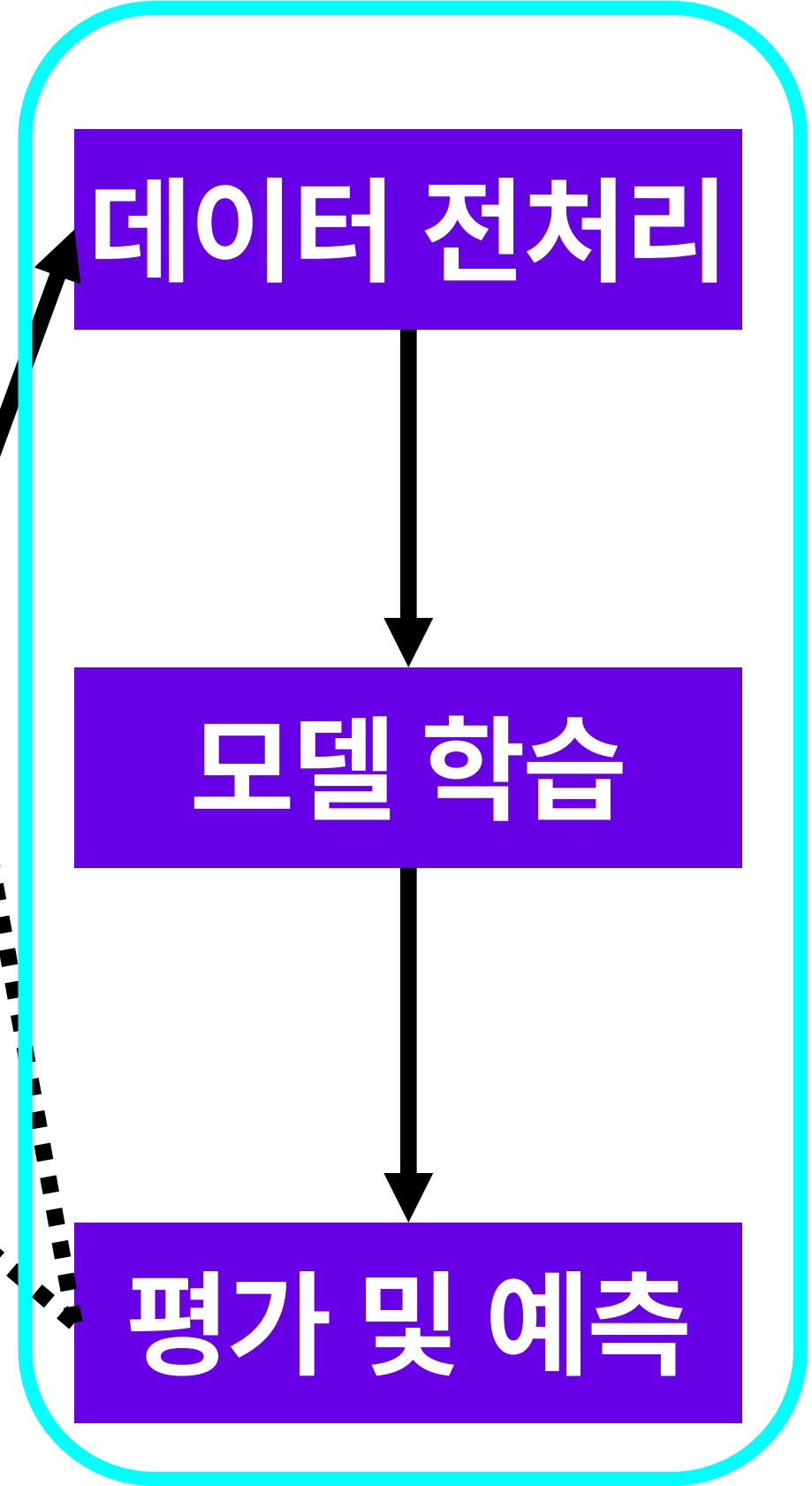


머신러닝 프로젝트 진행 과정

데이터 분석



머신러닝





데이터를 수집하고, 탐색적 데이터 분석을 거쳐 데이터를 확인하고 정제
EDA 결과를 바탕으로 모델 선택 과정을 통해 어떤 머신러닝 모델을 사용할지 결정



선택한 모델을 적용하기 위해 **데이터 전처리**, 이를 바탕으로 **모델학습** 수행
학습된 모델과 평가 데이터로 **예측**하여 모델의 성능을 **평가**
평가 결과가 배포하기 적합하다고 판정된다면 최종적으로 프로젝트를 배포



- 고품질 데이터를 수집해야 신뢰할 수 있는 모델을 만들 수 있음
- 데이터의 양과 다양성은 모델의 성능에 큰 영향을 미침
- 데이터베이스, 크롤링, 공공데이터 등을 이용해 수집



- EDA를 통해 데이터의 특성과 패턴을 이해
- 데이터의 특성을 이해함으로써 명확한 문제 정의와 목표 설정



- 다양한 모델을 평가하고 최적의 성능을 갖는 모델을 선택
→ 예측 정확도 극대화
- 모델의 일반화 능력 확보와 과적합 방지



- 선택한 모델에 적합한 학습용 데이터로 처리하는 과정
- 전처리를 통해 높은 품질의 데이터 확보
- 모델의 일관된 데이터 처리를 도움



전체 데이터세트를 필요한 부분에 맞게 나누는 과정

- 일반적으로 훈련세트와 테스트세트로 분리
- 전체 데이터세트를 모델 학습에 모두 사용할 경우 모델의 일반화 성능을 평가할 수 없음
- 학습 데이터로 평가 시 과적합으로 좋은 성능을 보일 수 있음



```
from sklearn.model_selection import train_test_split

feature_data = np.random.randint(0, 10, 50)
label_data = np.random.randint(0, 2, 50)

feature_train, feature_test, label_train, label_test = train_test_split(feature_data,
                                                                           label_data,
                                                                           test_size = 0.3,
                                                                           random_state = 42)
```

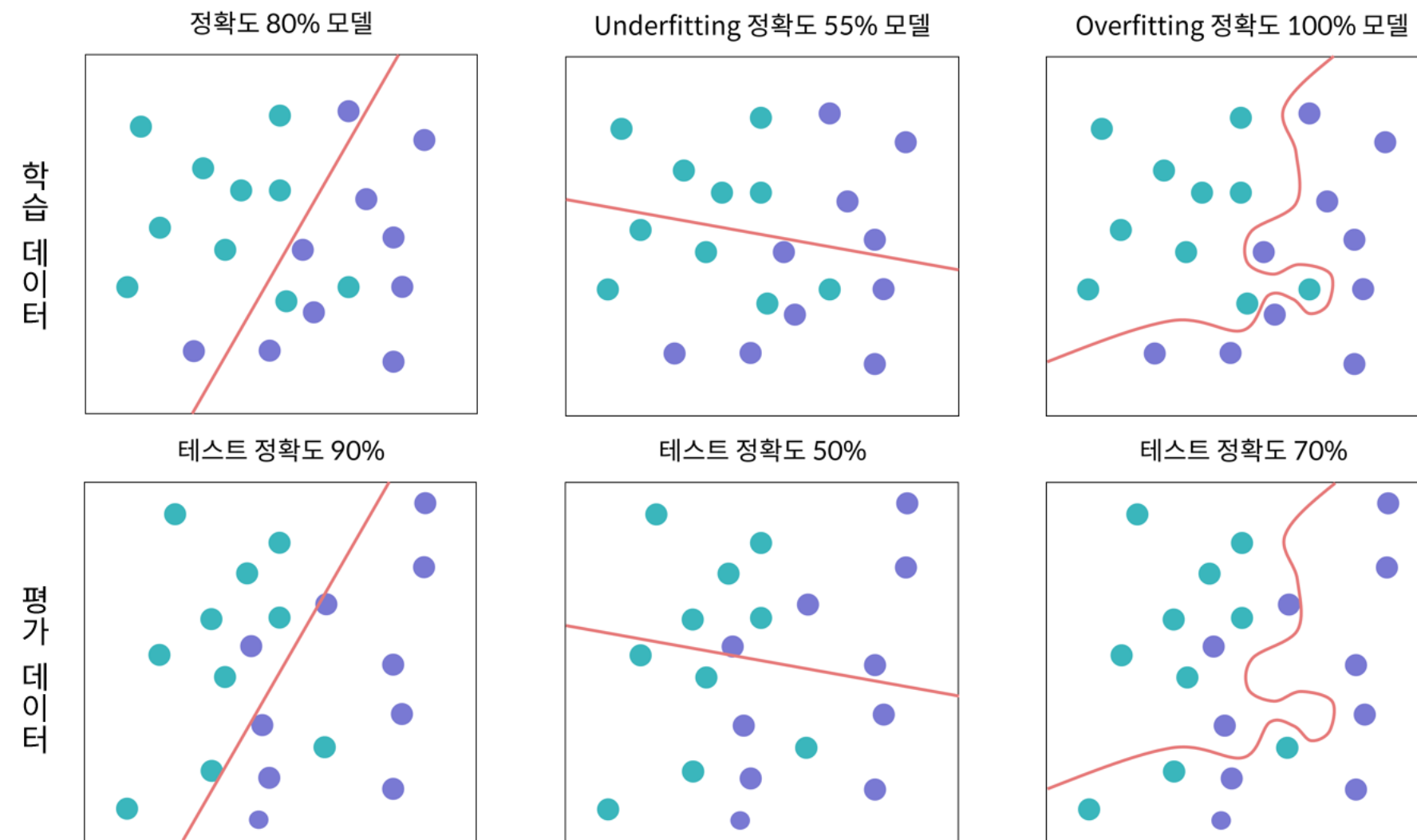
- **train_test_split(*arrays, test_size, random_state)**
 - 전체 데이터셋을 훈련 데이터셋과 테스트 데이터셋으로 나누어주는 함수
 - *arrays = 분리한 데이터 세트
 - test_size = 전체 데이터셋에 대한 테스트 데이터셋의 비율
 - random_seed = 동일한 결과를 위한 난수값



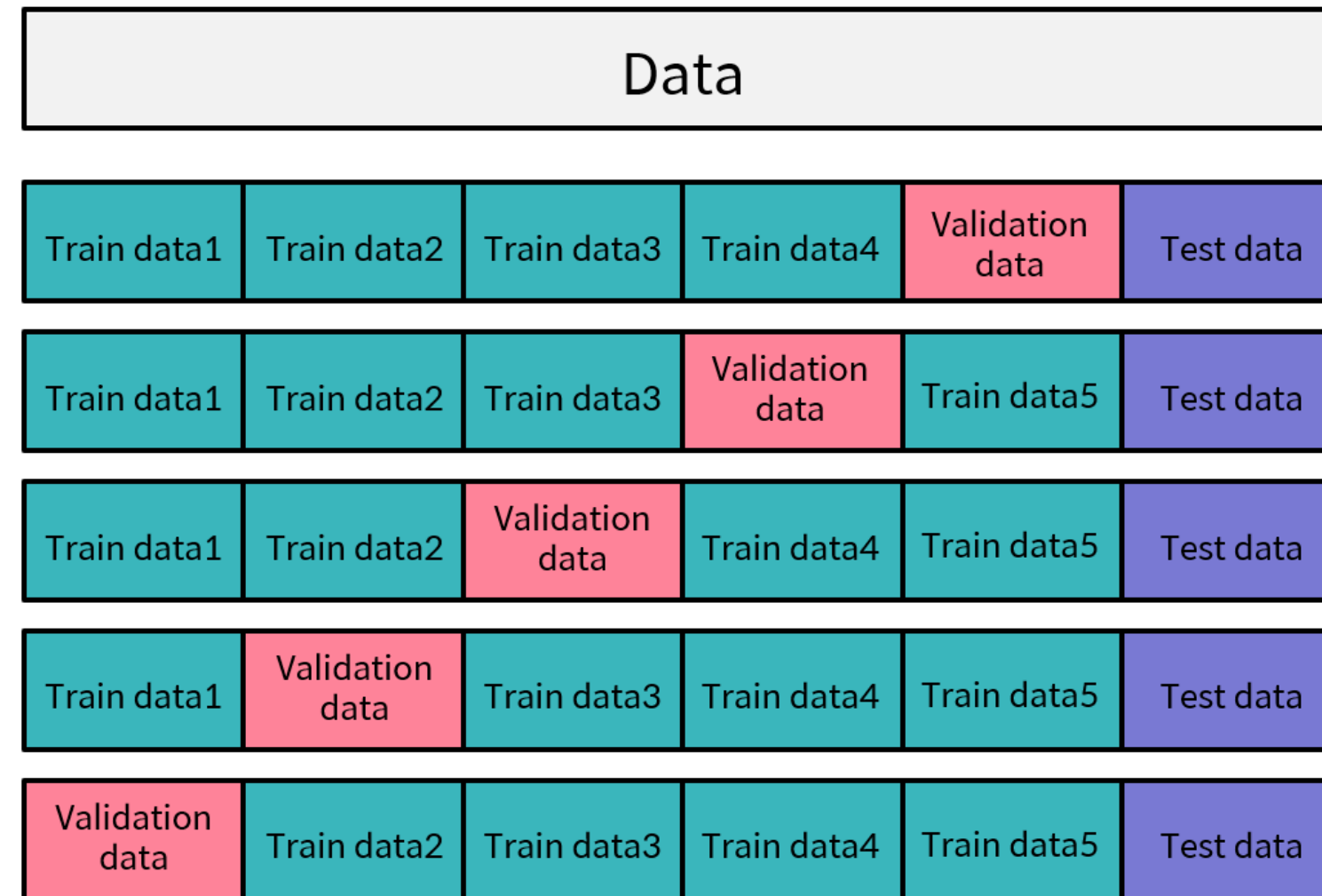
- 데이터의 패턴을 학습하여 예측 성능을 높이는 과정
- 모델이 새로운 데이터에도 잘 작동하도록 일반화하는 것이 중요



- 성능 평가를 통해 모델의 신뢰성 확인
- 평가를 통해 모델의 약점을 파악하고 개선
- 모델 평가 방법은 모델의 유형과 특정 작업에 따라 달라짐 (F1, MSE, MAE, R^2)



모델이 데이터에 **과하게 학습된 현상**



- **K-fold 교차 검증**
- 과적합을 피하기 위해 사용되는 가장 대표적인 교차 검증 방법
- 학습 데이터를 고정된 형태의 학습 데이터로 사용하지 않도록 변화시키는 방법



가설 검증이나 모델을 적용하기 전

데이터 스스로에 대한 정보를 전달하도록 만드는 방법

- 데이터를 이해하기 위한 단계
- 탐색 과정을 거쳐 결국 데이터를 표현하는 적절한 모형, 시각화 산출물, 다음 과정을 위한 데이터를 생성



- 결측치 처리
- 이상치 처리

- 스케일링
- 바이닝
- 더미



- 변수 확인
 - 데이터를 구성하고 있는 변수의 자료형과 의미하는 바 파악
- Raw data 확인
 - 변수 하나에 대한 통계적 특성 확인 단계
 - 평균, 최빈값, 중간값 등 각 변수들의 분포 확인



데이터 확인 구현

```
df = sns.load_dataset('titanic')
```

```
df.info()
```

```
df.shape
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 891 entries, 0 to 890
```

```
Data columns (total 15 columns):
```

#	Column	Non-Null Count	Dtype
0	survived	891 non-null	int64
1	pclass	891 non-null	int64
2	sex	891 non-null	object
3	age	714 non-null	float64
4	sibsp	891 non-null	int64
5	parch	891 non-null	int64
6	fare	891 non-null	float64
7	embarked	889 non-null	object
8	class	891 non-null	category
9	who	891 non-null	object
10	adult_male	891 non-null	bool
11	deck	203 non-null	category
12	embark_town	889 non-null	object
13	alive	891 non-null	object
14	alone	891 non-null	bool

```
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
```

```
memory usage: 80.7+ KB
```

(891, 15)

- **.info()**

- 변수의 데이터 타입과 개수를 확인하는 함수

- **.shape**

- 데이터 전체 크기를 확인하는 함수



데이터 확인 구현

```
df = sns.load_dataset('titanic')  
  
df.head()  
df.tail()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
886	0	2	male	27.0	0	0	13.00	S	Second	man	True	NaN	Southampton	no	True
887	1	1	female	19.0	0	0	30.00	S	First	woman	False	B	Southampton	yes	True
888	0	3	female	NaN	1	2	23.45	S	Third	woman	False	NaN	Southampton	no	False
889	1	1	male	26.0	0	0	30.00	C	First	man	True	C	Cherbourg	yes	True
890	0	3	male	32.0	0	0	7.75	Q	Third	man	True	NaN	Queenstown	no	True

- **.head()**

- 데이터의 상위 5개 행을 출력해주는 함수

- **.tail()**

- 데이터의 하위 5개 행을 출력해주는 함수



- 결측치 처리
 - 누락된 데이터가 있는 상태일 경우 모델이 왜곡될 수 있음
- 이상치 처리
 - 데이터와 동떨어진 관측치로, 모델이 왜곡될 수 있음
 - 변수의 분포 시각화(boxplot, histogram 등)를 통해 찾아낼 수 있음



- 새로운 관측치나 변수를 추가하지 않고 기존의 데이터를 보다 유용하게 만드는 방법
- 스케일링 : 데이터의 단위나 분포 변경
- 바이닝 : 연속형 데이터를 범주형 데이터로 변환
- 더미 : 범주형 데이터를 연속형 데이터로 변환



데이터셋에서 누락된 값

- 데이터 수집과정에서 발생할 수 있음
- 머신러닝 **모델의 성능에 부정적인 영향**을 줌



결측치 확인하기

```
df = sns.load_dataset('titanic')  
  
df.isnull().sum()
```

survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0
embarked	2
class	0
who	0
adult_male	0
deck	688
embark_town	2
alive	0

- **isnull()**

- DataFrame 내의 결측치를 확인하여 bool 형식으로 반환해주는 함수

- **isnull().sum()**

- isnull() 함수와 sum() 함수를 함께 사용시 각 변수의 결측치 개수를 반환



- 삭제
 - 전체 삭제 (열 삭제)
 - 부분 삭제 (행 삭제)
- 대체
 - 일괄 대체
 - 유사 유형 대체



결측치 처리 – 전체 삭제

```
df = sns.load_dataset('titanic')  
  
df.drop(['deck'], axis=columns, inplace=True)
```

- 특정 열에 결측치가 많이 포함되어 있는 경우
- **.drop(labels, axis, inplace)**
 - 데이터를 삭제하는 함수
 - labels = 제거할 데이터 설정
 - axis = 축 설정 (rows 또는 columns)
 - inplace = 원본데이터 수정 여부



결측치 처리 – 부분 삭제

```
df = sns.load_dataset('titanic')  
  
df.dropna(inplace=True)
```

- 데이터가 비교적 많고 결측치가 적은 경우
- **.dropna(inplace)**
 - 결측치가 있는 행을 삭제해주는 함수
 - inplace = 원본데이터 수정 여부



결측치 처리 - 일괄 대체

```
df['age'].mean()    # 평균  
df['age'].mode()    # 최빈값  
df['age'].median()  # 중간값
```

29.69911764705882

24.0

28.0

- 다른 관측치의 **평균, 최빈값, 중간값** 등으로 대체
- 일반적으로 **데이터의 개수가 적은 경우** 사용
- 자칫 의미 없는 값을 사용하게 되면 **데이터의 왜곡**이 일어나므로 신중하게 사용해야 함



```
df['age'] = df['age'].fillna(df['age'].mean())  
  
df['age'] = df['age'].fillna(df['age'].mode())  
  
df['age'] = df['age'].fillna(df['age'].median())
```

- **.fillna(value, inplace)**
 - 결측치를 주어진 값으로 대체해주는 함수
 - value = 결측치에 대신하여 넣어줄 값을 지정
 - inplace = 원본 데이터 수정 여부



머신러닝을 사용해 결측치가 있지 않은 다른 데이터를 학습하고 이를 바탕으로 결측치를 예측하는 방식

- 회귀 모델, K-최근접 이웃 등의 머신러닝 모델을 사용

데이터셋에서 다른 데이터 포인트와 크게 벗어난 값

	Height	Weight	Age
0	170	70	25.0
1	165	65	30.0
2	180	80	28.0
3	175	75	22.0
4	160	55	-5.0
5	168	60	32.5
6	172	68	27.0

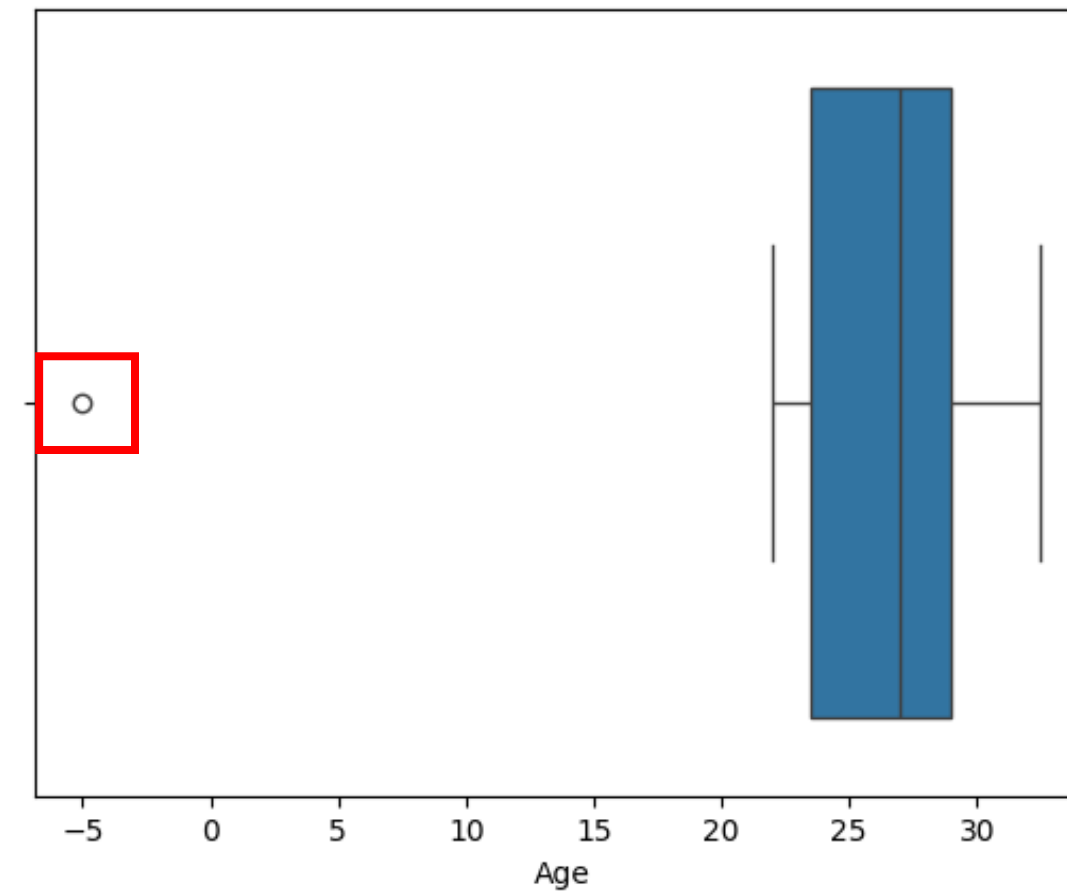
- 데이터 수집과정에서 발생할 수 있음
- 머신러닝 모델을 왜곡할 수 있음
- 이상치를 판단하는 기준은 여러가지가 있음



이상치 확인

```
import seaborn as sns
import matplotlib.pyplot as plt

# 상자 그림
sns.boxplot(x=df['Age'])
plt.xlabel('Age')
plt.show()
```



- seaborn 라이브러리 내 boxplot 함수를 사용하여 이상치를 확인할 수 있음
- 연속형 변수의 데이터분포를 확인하기 좋음
- **boxplot(x)**
 - 데이터를 상자그림으로 시각화해주는 함수
 - x = 시각화할 열



이상치 처리

- **단순 삭제**

- 이상치가 Human error에 의해 발생한 경우 해당 관측치 삭제
- 예) 나이값 중 소수가 있는 경우

- **대체**

- 절대적인 데이터의 양이 적을 경우, 삭제 방법은 데이터 부족 문제를 야기
- 이상치에 해당하는 값에 대체값 입력

- **변수화**

- 이상치가 자연 발생한 경우
- 이상치만의 유의미한 정보가 있다고 판단하여 변수화하여 활용하는 방식



이상치 처리 - 단순 삭제

```
df = df[(df['Age'] > 0) & (df['Age'] == df['Age'].astype(int))]
```

	Height	Weight	Age
0	170	70	25.0
1	165	65	30.0
2	180	80	28.0
3	175	75	22.0
6	172	68	27.0

- 데이터프레임 내 Age 변수에 음수와 소수는 이상치에 해당하므로 삭제
- 대괄호 내에 **조건**을 넣어 처리
- **.astype()**
 - 데이터형을 바꿔주는 함수
 - 원하는 데이터형을 괄호 안에 전달



이상치 처리 - 대체

```
# 음수와 소수 값을 대체할 중앙값 계산
```

```
median_age = df[(df['Age'] > 0) & (df['Age'] == df['Age'].astype(int))]['Age'].median()
```

```
# 음수와 소수 값을 중앙값으로 대체
```

```
df['Age'] = df['Age'].apply(lambda x: median_age if x <= 0 or x != int(x) else x)
```

	Height	Weight	Age
0	170	70	25.0
1	165	65	30.0
2	180	80	28.0
3	175	75	22.0
4	160	55	27.0
5	168	60	27.0
6	172	68	27.0

- 데이터프레임 내 Age 변수에 음수와 소수는 이상치에 해당하므로 삭제
- 대체할 값을 구한 뒤, 이상치에 해당하는 값들을 찾아 적용



이상치 처리 - 변수화

```
# 이상치 판단 함수
def is_outlier(age):
    return age <= 0 or age != int(age)

# 이상치 여부를 표시하는 새로운 컬럼 추가
df['Age_Outlier'] = df['Age'].apply(is_outlier)
```

	Height	Weight	Age	Age_Outlier
0	170	70	25.0	False
1	165	65	30.0	False
2	180	80	28.0	False
3	175	75	22.0	False
4	160	55	-5.0	True
5	168	60	32.5	True
6	172	68	27.0	False

- 이상치에 해당되는 값들을 확인하는 함수 생성
- 새로운 변수에 이상치 여부를 저장



기존의 데이터를 보다 유용하게 만드는 방법

- 특성 생성
- 특성 선택
- 특성 변환



특성 엔지니어링 - 특성 생성

- 기존 데이터를 기반으로 **새로운 특성을 생성**하는 방식
- 날짜 처리 및 시간 처리
 - 날짜를 년, 월, 일, 요일 등으로 분리하거나 시간 차이 계산
- 텍스트 데이터 처리
- 상호작용 특성
 - 키와 몸무게를 이용해 BMI 계산



특성 생성 - 날짜 처리

```
data = {  
    'Date': ['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04', '2023-01-05']  
}
```

```
df = pd.DataFrame(data)
```

```
# split 함수를 사용하여 날짜를 분리
```

```
df[['Year', 'Month', 'Day']] = df['Date'].str.split('-', expand=True)
```

	Date	Year	Month	Day
0	2023-01-01	2023	01	01
1	2023-01-02	2023	01	02
2	2023-01-03	2023	01	03
3	2023-01-04	2023	01	04
4	2023-01-05	2023	01	05

- 날짜를 년, 월, 일, 요일 등으로 분리하여 새로운 변수로 저장
- **.str.split(pat, expand)**
 - pat = 문자열 분할에 사용할 구분자 지정
 - expand = 분할된 문자열을 데이터프레임의 열로 확장시킬지 여부



특성 엔지니어링 - 특성 선택

- 모델에 **유용한 특성은 선택**하고, **중요하지 않은 특성은 제거**하는 방식
- 상관 분석
 - 특성 간의 상관 관계를 분석하여 중복되는 특성 제거
- 특성 중요도
 - 모델 기반 특성 중요도를 사용해 특성 선택
- 차원 축소
 - 차원 축소 기법을 이용해 특성의 수 줄이기



특성 엔지니어링 - 특성 변환

- 데이터 스케일링, 인코딩, 차원 축소 등의 방법을 사용해 특성을 변환
- **스케일링**
 - 데이터에 log 함수를 적용하거나 square root 등을 사용하여 데이터 분포를 변환
- **바이닝**
 - 범주형 데이터가 필요한 모델인 경우, 연속형 데이터를 범주형 데이터로 변환
- **더미**
 - 연속형 데이터가 필요한 모델인 경우, 범주형 데이터를 연속형 데이터로 변환



특성 변환 - 스케일링

```
df = sns.load_dataset('titanic')

# 로그 변환을 위해 0 또는 음수 값을 처리
# (예: 최소값을 더하거나, 음수 값만 변환하지 않도록 처리)
df['fare'] = df['fare'].apply(lambda x: np.log(x + 1) if x > 0 else 0)
```

0	0.746789
1	1.454092
2	0.783379
3	1.384000
4	0.789713
	...
886	0.970422

- fare 열의 모든 데이터를 로그값으로 변환
- **log(x)**
 - 주어진 x에 대한 로그값을 계산해주는 함수



특성 변환 - 바이닝

```
df = sns.load_dataset('titanic')

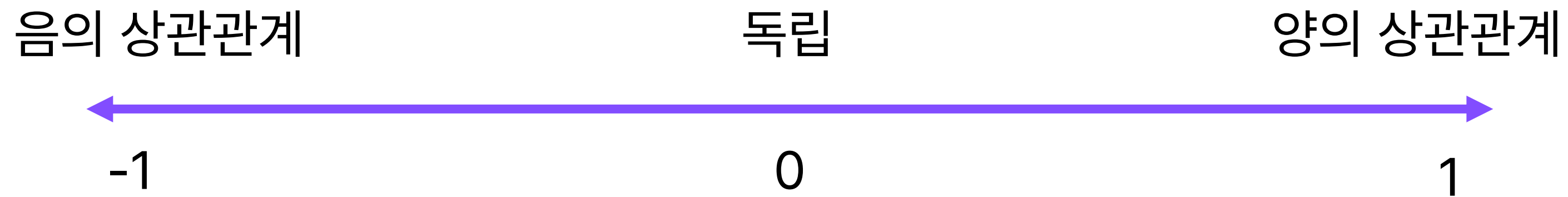
df.loc[df['age'] <= 20, 'age'] = 0
df.loc[(df['age'] > 20) & (df['age'] <= 40), 'age'] = 1
df.loc[(df['age'] > 40) & (df['age'] <= 60), 'age'] = 2
df.loc[df['age'] > 60, 'age'] = 3
```

	survived	pclass	sex	age	sibsp
0	0	3	male	1.0	1
1	1	1	female	1.0	1
2	1	3	female	1.0	0
3	1	1	female	1.0	1

- age의 데이터를 구간 별로 나누어 값을 부여
 - ~ 20 = 0
 - 21 ~ 40 = 1
 - 41 ~ 60 = 2
 - 60 ~ = 3



두 변수 간의 관계를 나타내는 통계적 개념



- 한 변수의 변화가 다른 변수의 변화와 어떻게 관련되어 있는지 측정
- -1 에서 1 사이의 값을 가짐



데이터 간의 상관관계를 시각화 하여 분석하고 결과를 바탕으로 머신러닝 모델 선택의 방향 제시

- **pairplot**
 - 변수 간 산점도 출력
- **heatmap**
 - 피어슨 상관 계수를 사용하여 변수 간의 상관관계 분석



상관관계 분석 - 피어슨 상관계수

	Height	Weight	Age
0	187.640523	98.247260	53
1	174.001572	49.783614	27
2	179.787380	50.942725	31
3	192.408932	84.540951	24
4	188.675580	52.403149	56

```
pearson_corr = df.corr(method='pearson')
```

	Height	Weight	Age
Height	1.000000	0.111729	-0.110190
Weight	0.111729	1.000000	0.053676
Age	-0.110190	0.053676	1.000000

- **.corr(method)**

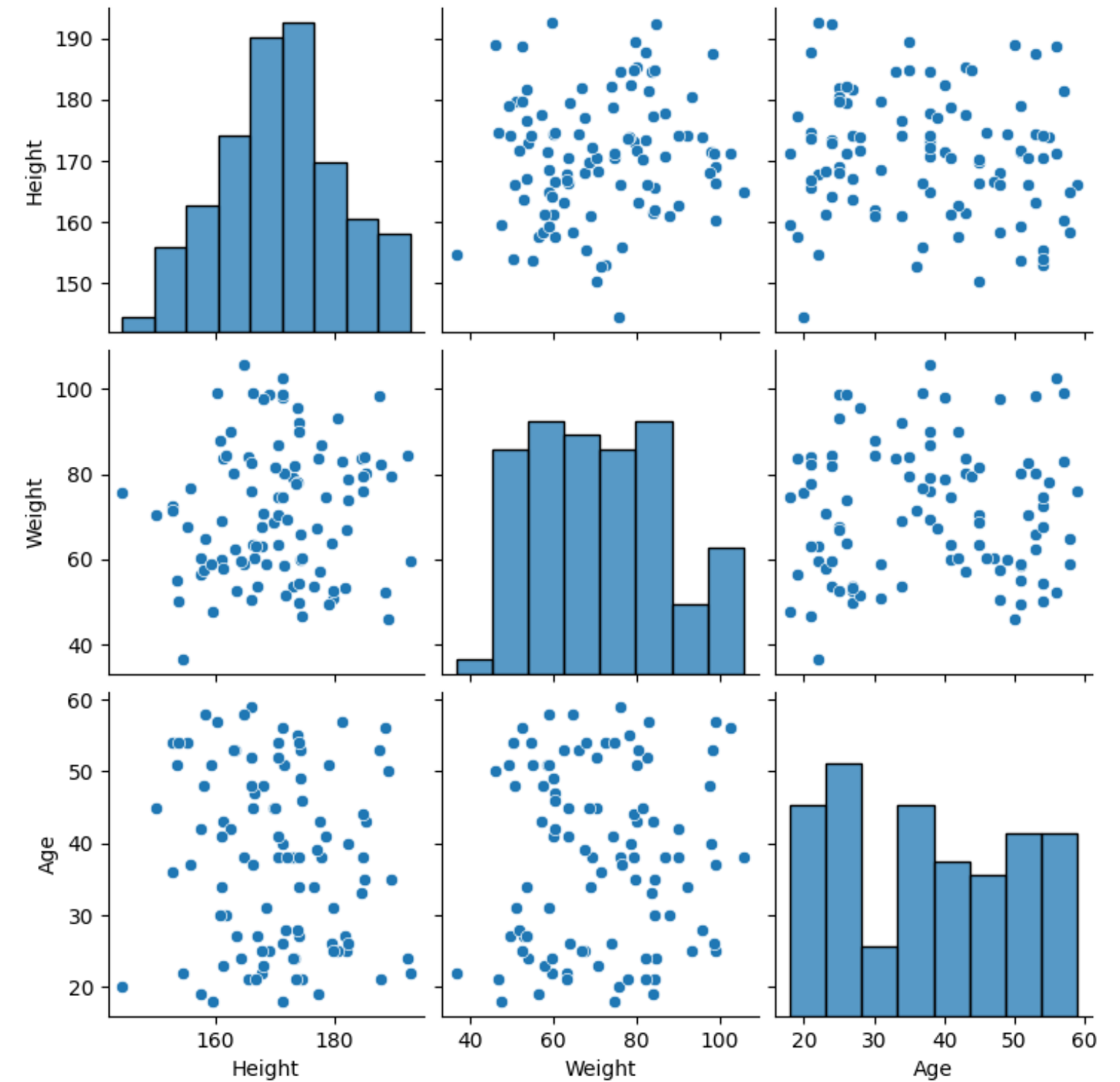
- 상관계수 값을 구해주는 함수
- method = 적용할 상관계수 방식
- pearson, kendall, spearman 이 있음



상관관계 분석 – pairplot

```
# pairplot 그리기  
sns.pairplot(df)  
plt.show()
```

- **.pairplot(data)**
 - 변수 간 산점도를 그려주는 함수
 - 데이터 프레임 전체 또는 분석을 원하는 변수명을 전달





상관관계 분석 – heatmap

```
pearson_corr = df.corr(method='pearson')  
  
# heatmap 그리기  
sns.heatmap(pearson_corr, annot=True)  
plt.show()
```

- **.heatmap(corr, annot)**
 - 상관계수를 이용해 시각화 해주는 함수
 - corr = 변수 간의 상관계수 값
 - annot = 그림 내 상관계수 값 표시 여부

